

The Aniketos Design-Time Framework Applied – a Case in Air Traffic Management

Stéphane Paul¹, Alessandra Tedeschi², Erlend Andreas Gjære³, Ivonne Herrera³

¹Thales Research and Technology, Avenue Augustin Fresnel, 91767 Palaiseau, France
stephane.paul@thalesgroup.com

²Deep Blue, Piazza Buenos Aires, 20, 00198 Roma, Italy
alessandra.tedeschi@dblue.it

³SINTEF ICT, Strindveien 4, NO-7465 Trondheim, Norway
{erlendandreas.gjare, ivonne.a.herrera}@sintef.no

Abstract. In order to assess the industrial relevance of the Aniketos design-time framework, we report on its application to a typical use-case of the Air Traffic Management (ATM) System Wide Information Management (SWIM) system: a meteorological information request from a pilot in an aircraft involving air-ground data-link communications. The scope of the study runs from security requirements elicitation, to the deployment of dummy services implementing the meteorological request secured process and communication functions. The evaluation shows that a rich set of security requirements can be captured and managed throughout the design-time engineering process. The Aniketos design-time framework is assessed as a sound baseline. To allow for an industrial exploitation follow-up, some required improvements have been proposed.

Keywords: Evaluation, Design-Time, Security Engineering, Tools, Case-Study, Air Traffic Management, System Wide Information Management

1 Introduction

The European airspace is fragmented and congested. Air Navigation Services and their supporting Air Traffic Management (ATM) systems are not fully integrated, and are based on technologies already running at their max. To cope with this congestion, early this century, the ATM community thought that a paradigm shift (i.e., a breakthrough) was required. This led to the creation of the Single European Sky ATM Research (SESAR) Programme. The SESAR Joint Undertaking (SJU) was created under European Community law to manage the SESAR Development Phase. During the recent years, a new operational concept (CONOPS) was developed for ATM. One of the main identified operational enablers is the System Wide Information Management (SWIM). SWIM is a distributed processing environment, which replaces data level interoperability and closely coupled interfaces with an open, flexible, modular and secure data architecture, totally transparent to users and their applications.

SWIM will be open to all traditional ATM stakeholders and systems. However, it is also planned to be open to non-traditional ATM stakeholders, thus giving birth to

new and strong security needs in a domain that has always focused exclusively on safety. This makes SWIM a very relevant case for applying and assessing the new Aniketos security engineering design-time processes, methods and tools.

In order to assess the industrial relevance of the Aniketos design-time framework, we report on its application to a typical SWIM use-case: a meteorological information request from a pilot in an aircraft. Section §2 provides an overview of all the Aniketos design-time security engineering evaluation activities that were run. Another perspective is presented in Chapter 16: Supporting Security and Trust in Complex e-Government Services.

On our selected case-study (cf. section §3), we applied the overall Aniketos design-time process. The first step was the capture and documentation of the security requirements (cf. §4) using the Socio-Technical Security (STS) modelling method, language (cf. Chapter 5) and tool (cf. Chapter 7). This was compared to the existing list of security requirements, as extracted from the SESAR documentation [1-9].

Then, we realised a shift from the SWIM problem space to the SWIM solution space, by modelling the corresponding business processes (cf. §5), integrating the security measures along the path, and developing the corresponding web services for deployment. For this work, we used the Service Composition Framework (SCF) tool, which integrates interaction with several components and services developed in Aniketos (cf. Chapter 4, Chapter 6, and Chapters 8 to 11). This engineering work with Aniketos tools was briefly compared to the current industrial best practices.

2 Methodology

The evaluation activities and assessments presented in sections §4 and §5 are the result of many focused evaluation sessions carried out throughout the whole Aniketos project lifespan. These evaluation activities can be classified in three main categories:

- in-depth specific evaluation activities, to test and assess individual components, performed by the authors of this chapter, acting as Aniketos end-users (cf. §2.1);
- on-the-job evaluations, involving third-party operational and security experts, to collect detailed external feedback on methodologies and tools (cf. §2.2);
- presentations to ATM stakeholders and potential end-users, to collect high-level feedback and assess general applicability in an industrial context (cf. §2.3).

2.1 Single Components Evaluations

Single components evaluations were run as an iterative process. A first evaluation session was dedicated to the investigation of modelling practices of STS-tool users, in order to better understand the modelling flows, and develop the STS methodology. Then, a workshop was organised in order to test and assess the STS-tool in its early implementation stage and inform further developments. An interactive validation session was organised to evaluate the Module Transformation Module (MTM) in its

final version. Finally, a long evaluation session was dedicated to SCF and its satellite tools.

The SWIM case also informed the evaluation and re-design of the existing supporting material, tutorial and guidelines for the Aniketos design-time tools. The availability of a challenging industrial scenario used as test-bed provided useful insights and feedback to the Aniketos tool developers. All comments were not taken into consideration due to project time and budget constraints.

2.2 On-the-Job Evaluation

The on-the-job evaluation was a one-shot experiment with three third-party SWIM operational and security experts. The main objective of this assessment was to provide detailed industrial feedback on the STS methods and tools. The method used for the assessment was simply to build and run a SWIM case study.

2.3 Presentations to ATM Stakeholders

In order to evaluate the impact of the design-time Aniketos solutions for the ATM domain, various activities were proposed for presenting Aniketos project results to ATM stakeholders:

- presentations to SESAR partners during security-related events and SESAR meetings were iteratively carried out, in order to gather feedback from relevant external stakeholders and refine the Aniketos results;
- the SWIM case study partners organised presentations and modelling sessions inside their own organisation with other departments working in the ATM domain;
- a final combined demonstration and evaluation workshop with ATM stakeholders (about 15 participants) was conducted in Rome on October 23rd, 2013.

The main aim of these activities was to present and evaluate the Aniketos design-time support tools for the ATM domain, with a focus on: (i) assessing the overall suitability of the Aniketos design-time solutions for the ATM domain and its compliance with existing methods, standards, procedures and work practices; (ii) evaluating the acceptability of the Aniketos design-time solutions for ATM practitioners, including how understandable, easy-to-use and effective they are in real operational and industrial contexts.

3 The System Wide Information Management Case

According to SESAR [4], the set of initial Air/Ground (A/G) SWIM applications includes: (i) the sharing of weather forecasts and aircraft weather observations for the reduction of impacts of hazardous weather; (ii) the sharing of traffic situation for reduction of impacts of traffic congestion; (iii) the sharing of temporary aeronautical data and collaborative diversion planning to improve the management of unexpected situations. The flight crew should be able access the data at any time (regardless of

the flight phase, airspace or communication link used), instantly (seamless loading without a long wait before information gets displayed) and will be confident that the data originates from a safe source approved by Regulation Authorities. The A/G SWIM aims to be a fast and secure in-flight connection, which will enable the flight crew to share and download data in real time, in large volumes and only from authorized sources.

The baseline scenario retained for Aniketos is an aircraft requesting weather information via SWIM. Thus, the specific A/G SWIM service considered is Weather Avoidance. Through the Weather Avoidance service the A/G SWIM can provide: (i) weather and turbulence forecasts; (ii) current weather situation for areas beyond the range of aircraft sensors and on-board weather radar; data may include the graphical shape of a large hazardous area, or textual information on the weather situation at alternative diversion airports.

We consider an aircraft flying its en-route Business Trajectory [10]. Weather conditions are changing and the pilots make a data-link request for *Meteorological Information (MET Info)* provision via SWIM. The aircraft *Airborne Broker* calls an intermediary *Ground Broker*. The *Ground Broker* enriches the information and provides the most recent *MET Info* to the *Airborne Broker*. This *MET Info* is timely stored in the SWIM by a *MET Service Provider* to be exploited by all the *MET Service Subscribers*.

Table 1. Excerpt of Functional Security Requirements from the SESAR programme

Id	Functional Security Requirement	Rationale
AG-SR1	Only publishers authorized for given information category shall be able to publish information of this category.	Data must be provided by the authority responsible for producing it.
AG-SR2	Only consumers authorized for given information category shall be able to receive information of this category.	Data must be received by authorised customer for a specific category.
AG-SR3	Unauthorized information access shall be prevented.	Security requirement, derived by Risk Assessment.
AG-SR4	The air/ground information management shall be able to provide information integrity of 0,999999 (i.e. probability of undetected error 10 ⁻⁶).	Most stringent requirement from the AIS/MET SPR, EUROCAE ED-175 / RTCA DO-324. No safety conditions.
AG-SR5	The air/ground information management shall be able to protect authenticity of the transferred information.	Security requirement, derived by Risk Assessment
AG-SR6	The air/ground information management shall be able to provide confidentiality of the transferred information.	Security requirement, derived by Risk Assessment.

AG-SR7	The air/ground information management shall fulfil security requirements for segregation of services between the Aircraft Control Domain (ACD), the Aircraft Information System Domain (AISD) and the Passenger Information and Entertainment System Domain (PIESD) - see ARINC 664 Part 5.	Security requirement, derived by Risk Assessment.
--------	---	---

In **Table 1**, we report some of the SESAR Functional Security Requirements that are relevant to our case-study. These requirements have been used as a reference through the whole Aniketos design-time security engineering process.

4 Eliciting the Security Requirements with STS

This section describes the activities performed using the Socio-Technical Security modelling language (cf. Chapter 5) and tool (cf. Chapter 7) to elicit and capture the security needs of our ATM SWIM case (cf. §3). The modelling work was done following the STS methodology [11]. This section also provides some initial assessment of the framework.

4.1 Study of the Context

The first step of the STS methodology is the “Study of the Context”. The objective is to set the scene, from an organisational, technical and operational view point. This step aims for breadth, not depth. The result should be short and easy to grasp.

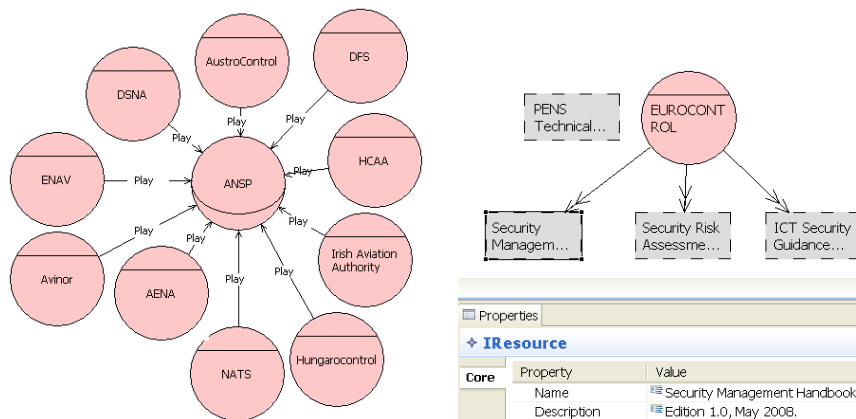


Fig. 1. Setting the scene: (a) roles and agents; (b) applicable standards and regulations

This step was particularly challenging in our ATM SWIM case-study. Indeed, many components of the considered ATM System of Systems (SoS) already exist, in terms of deployed and operated equipment, people and procedures. Building a relevant so-

cio-technical abstraction adequate to reason about SWIM security requirements from the extremely complex SoS and very detailed legacy implementations described in hundreds of different documents, was very far from trivial.

Our first approach was to model common knowledge about the ATM context, including, for example, a fine-grained description of the actors at play (cf. **Fig. 1a**) and the way they could impact and / or influence the system-under-study through the modelling of the standards and / or regulation that they impose (cf. **Fig. 1b**). The STS tool proved quite efficient in capturing this common knowledge.

However, for such a complex SoS as the overall European Air Traffic Management system, this approach proved completely inappropriate. Too many details about common knowledge facts meant that the key elements of the study were lost. As a consequence, the ecosystem non-participant stakeholders were not modelled individually in the STS-tool. Instead, they were captured as a unique rag-bag actor called SWIM Context, with the exception of:

- the *Pilot* requesting meteorological information;
- the *Aircraft System* supporting the interaction between the Pilot and the SWIM;
- the *Meteorological Service Provider* providing the requested meteorological information to the *Pilot* via the *SWIM technical infrastructure (TI)*.

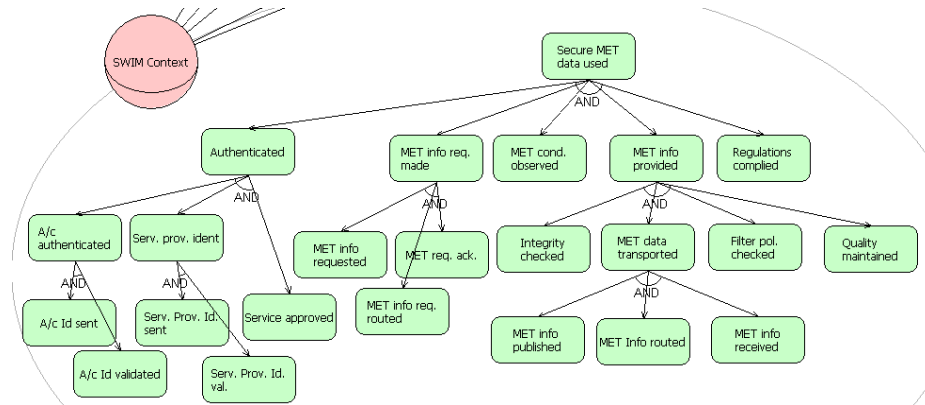


Fig. 2. Defining top-level goals

This original construct of the SWIM Context was key in defining the different actor goals. Indeed, a Pilot's goal to obtain fresh meteorological information, the SWIM TI's goal to securely route meteorological information requests and responses, and the Meteorological Service Provider's goal to provide accurate and timely meteorological information are all linked by some top-level soft-goals related, for example, to ensuring safe and efficient flights over Europe. Through the artificial SWIM Context construct, it was possible for us to define and decompose a top-level soft-goal related to the secure use of meteorological data (cf. **Fig. 2**) without too many discussions about who is at the origin of, or is responsible for, each sub-goal. This construct was also key in the following methodological steps, in particular when goal delegation was necessary (see below).

By contrast, the SWIM ecosystem participant stakeholders were modelled in more details, namely we chose to model individually: (i) a generic SWIM Access Point (SAP); (ii) the SWIM Transport Services, and (iii) the SWIM aircraft On-board Message Broker. The goals of these actors are defined later (cf. §4.3), through the delegation of goals defined in the SWIM Context.

4.2 Study of the Assets

The second step of the STS methodology is the “Study of the Assets”. The objective of this step is to identify and describe all the assets relevant to the scope of the study, and capture their intrinsic¹ security needs.

This was done for all ecosystem participant actors. For example, for the *SWIM Access Point*, the following assets were identified: (i) in terms of information assets: *Meteorological Information Request*, *Meteorological Information*, *Aircraft Identification*, and *Meteorological Server Identification*; (ii) in terms of goal assets: *Meteorological Information Request Routed*, and *Meteorological Information Routed*; (iii) in terms of document assets: local copies of the *Meteorological Information Request*, local copies of the *Meteorological Information*, local copies of the *Aircraft Identification*, and local copies of the *Meteorological Server Identification*. Then, the assets were described in detail, and linked together (cf. **Fig. 3**).

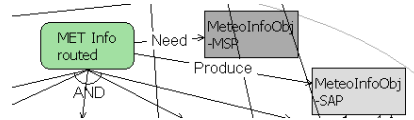


Fig. 3. Identifying assets and linking them together

The STS-tool allowed for the capture of quite a few intrinsic security requirements in a compact and formal way. For example, the model extract in **Fig. 4** shows that the *SAP* is only allowed to access *Meteorological Information* in order to distribute it, as a sub-task of the *MET Info routed* goal. This implies that the *SAP* is never allowed to view, modify or produce *Meteorological Information*, and the *SAP* may not distribute *Meteorological Information* for other purposes than routing it to the specified addressee.

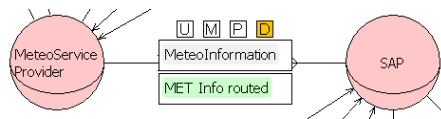


Fig. 4. Expressing a need-to-know and integrity requirement between the meteorological service provider and the system wide information management access point

¹ By intrinsic, we mean independent of the solution space, i.e. excluding any delegation / co-operation options; indeed, co-operation and delegation are the first steps in the solution space with respect to the problem space of top-level goal achievement by an actor.

Likewise, it was possible to capture some additional data availability and data transmission integrity requirements, but not all security needs could be easily expressed. To overcome these limitations, or to justify particular security requirements, the STS-tool allows for threats to be captured as threat events. Using the centralised Threat Repository Module (cf. Chapter 4 and Chapter 12), we searched for threats, by keyword (e.g. *impersonation*) and by business domain (e.g. *ATM*). We selected a few relevant threats and created new ATM-specific threats, which were stored in the Threat Repository. Although not being a replacement for proper risk analysis, the inclusion of threats in STS allowed explicating and sharing awareness on situations that are not otherwise described in the desired system. The STS-tool capability of propagating threat impacts throughout the model (cf. **Fig. 5**) further increased this security issue awareness. Beyond this shared awareness, threat modelling did not help us improve the remainder of the STS model, as one could have expected. This is probably because in our case, threat modelling was performed quite late in the modelling process. It is reasonable to think that our STS model had already been designed with these threats in mind. Threat modelling only allowed for making these threats explicit to all.

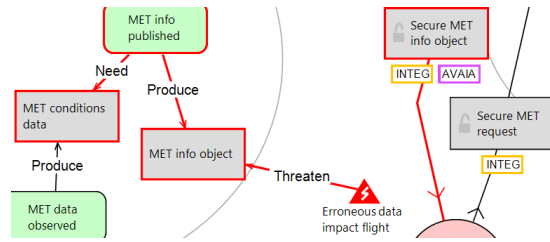


Fig. 5. Expressing threats with events, such as *Erroneous data impact flight*, allowing for the threat propagation analysis to be invoked (threatened paths and objects highlighted)

Although threat modelling was found to be quite appealing, we had to limit its usage because threats and their relations to actors and assets rapidly cluttered the diagram. A means to toggle threat visibility would have been of significant value.

Finally, let us recall that STS does not allow one to define scales for the security needs. This was found to be significantly detrimental in capturing security requirements.

4.3 Study of the Security Policies

The third step of the STS methodology is the “Study of the Security Policies”. The objective of this step is to extend the analysis of the security needs by considering how collaborative goals are fulfilled. Collaborative goals imply security relevant features, namely, the delegation of all or part of the work, and sharing resources to support the fulfilment of those goals. Thus, the analysis performed in this step extends the scope of the capture of the security needs (cf. §4.2) to the delegation of rights over

goals and to authorisations over data that are useful for the fulfilment of those collaborative goals.

In our case-study, we extensively used delegations between our *SWIM Context* and our three SWIM ecosystem participants (cf. §4.1). Delegation is one of the most powerful concepts of the STS modelling language (STS-ml). However, it is tricky to use and we found its semantics to be insufficiently defined. Typically, it remained unclear if goal delegation was a simple delegation of fulfilment of the goal, or a delegation of responsibility over the fulfilment of the goal, or a transfer of accountability. In our case-study, the actors interacting with the SWIM have no hierarchical relations² with the SWIM. Thus, neither of them can (legally) delegate a task to SWIM, and vice-versa. Typically, the Pilot and SWIM belong to independent organisations. When the Pilot uses SWIM to route a message to the Meteorological Service Provider, he does not (legally) delegate the routing, but he cooperates with a third-party, and he has to trust that third-party.

4.4 Conclusion

By contrast with traditional risk-based security requirement elicitation techniques (e.g., [12, 13, 14]), the Socio-Technical Security (STS) modelling approach of Aniketos is risk agnostic. Traditional risk-based security requirement elicitation techniques operate in the solution space, assessing the vulnerabilities of the designed solutions, in parallel to the assessment of the threats and impacts of the feared events. The STS modelling approach of Aniketos operates solely in the problem space, with a strong focus on business / operational goals. The intrinsic security needs can be elicited essentially based on the will to achieve those goals, whilst preserving the identified assets; the identification of feared events may help in this elicitation. As such the STS modelling approach of Aniketos is perfectly suited for a very early security engineering activity, during which business / operational staff can be easily involved, without introducing any complexity related to the solution space. For these reasons, the STS-ml and STS-tool were a perfect fit for the ATM SWIM case-study.

However, the current version of the STS-ml and STS-tool are not completely ready for industrial use. Based on our case-study, the main issues that would need to be solved beforehand include: (i) the impossibility of clearly defining the boundaries of the system-under-study; (ii) the impossibility of defining security need scales, in particular for the Confidentiality, Integrity and Availability (CIA) criteria; (iii) insufficiently formal semantics, in particular for organisation structures, goal delegation, asset ownership, and Role Based Access Control (RBAC).

² This is a typical situation in the context of a System of Systems (SoS).

5 Designing & Securing the SWIM Business Process with SCF

This section describes the activities performed using the Aniketos Service Composition Framework (SCF) to design and secure our ATM SWIM business process (cf. §3). This section also provides some initial assessment of the framework. For technical details relating to SCF and its satellite tools, please refer to Chapter 6 and Chapters 8 to 11.

5.1 Designing the (Unsecure) SWIM Business Process with SCF

The first step we carried out in running our ATM SWIM case-study was business process modelling without any consideration for security issues. We developed multiple BPMN diagrams, comprehending over thirty tasks. Some tasks were user tasks, e.g. *Login*, but most tasks were service tasks, e.g. *Authenticate*. SCF proved itself adequate for the job.

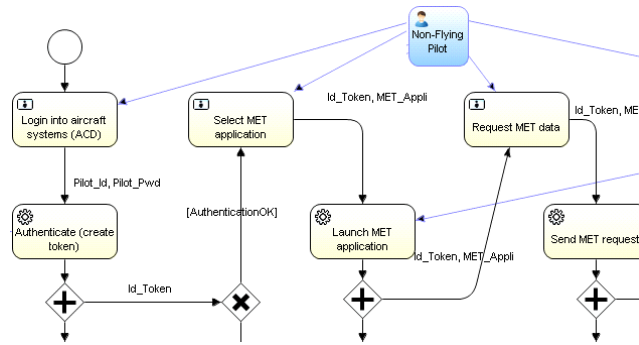


Fig. 6. Separating clearly user tasks from service tasks

Whilst the Business Process Modelling Notation (BPMN) specification allows for many types of tasks, including user tasks, the Aniketos service specifications consist of service tasks only. A service task is performed by a web service. A composite service process is hence an executable workflow that is realised by web services alone, and consumed as a web service itself. During the course of the case-study, this limitation proved to have significant but manageable impacts on our BPMN design. Some diagrams had to be redrawn to separate clearly user tasks from service tasks (cf. Fig. 6). For other processes with heavy human-machine interactions, this might however be a deterrent constraint.

5.2 Discovering Services: a Prerequisite to Security Specification

The next step in our engineering process should have been the securing of our business process. However, with the Aniketos engineering process, security specification on service compositions can only be done after atomic service discovery within the Service Composition Framework, in order to retrieve the atomic service signature.

This approach is clearly disputable since, in a real industrial context, most of these services may not yet exist.

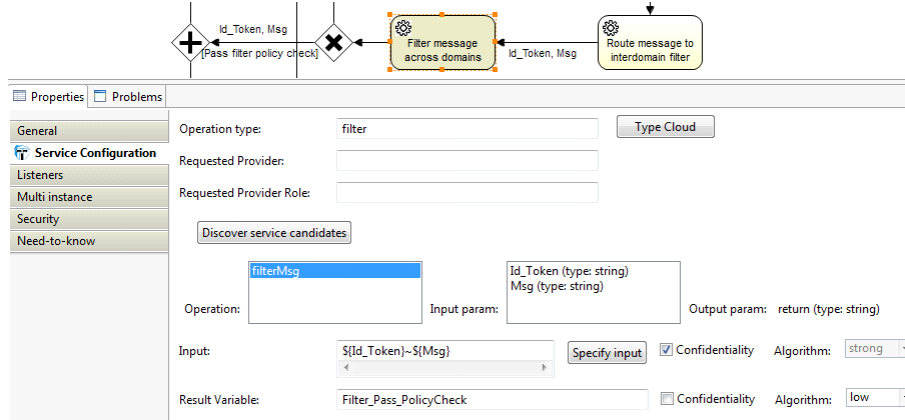


Fig. 7. Service discovery and confidentiality specification

To proceed with our case-study, we implemented, deployed and published 26 web services, using the Aniketos Marketplace. Most of these web services, e.g. *Log*, were dummy services doing nothing, but services that were key to observing the execution of the meteorological request-response scenario were effectively developed, together with their corresponding data flow interfaces and human-machine interfaces.

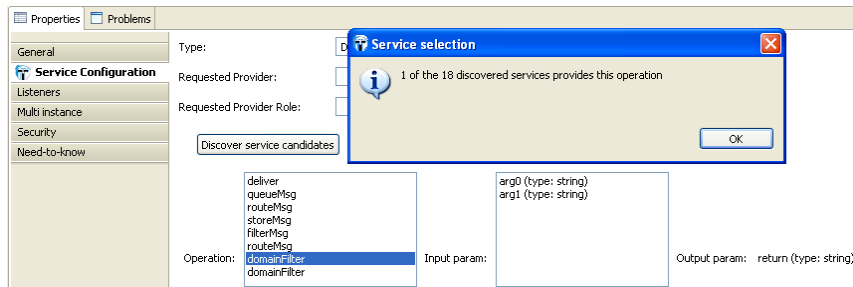


Fig. 8. Signature of composite service deployed and discovered with SCF

The first major issue we faced with service discovery in SCF was that the response to a discovery request was (only) a list of operations with their input and output parameters (cf. **Fig. 7**). This response is much shorter than the output that can be obtained from a search directly in the Aniketos Marketplace. Typically, the service names, the service descriptions, the service provider and the list of other tags were missing in SCF. An improvement of the Human-Machine Interface (HMI) is needed here, to avoid ambiguities in the identification of services.

The second major issue we faced with service discovery was related to method signatures. When a composite service is stored in and retrieved from the Aniketos

repository, the 1st parameter of the associated method is an array of the names of all the parameters, whilst the 2nd parameter is an array of the corresponding values, cf. **Fig. 8**.

This implementation for composite services was found to be inadequate for service discovery, because it reduces the signature of the composite service to the sole name of the method. Moreover, it is different from the signature used for atomic services, cf. **Fig. 7**. An improvement of the implementation is needed here to support an effective discovery.

5.3 Securing the ATM SWIM Business Process

Having defined the business process (cf. §5.1) and performed service discovery for all atomic services (cf. §5.2), it was now possible to secure the business process. Each security property that was addressed in our test case is discussed below.

Separation and binding of duty (SoD/BoD) are the most straightforward security properties that can be modelled using SCF, integrated nicely in its GUI and tools palette. It is supported through the SecureBPMN extension (cf. Chapter 8) to BPMN and the Activiti tool. Although the SESAR SWIM project does not list any SoD/BoD requirement, this construct was extensively used during our case-study modelling, cf. **Fig. 6**. However, the concrete syntax makes it impossible to express SoD/BoD properties across diagrams. For example, it was impossible to capture a binding of duty between the logging services in all our BPMN diagrams. Another issue with the graphical notation was scalability, with quickly rising visual clutter in the diagrams.

The screenshot shows a web-based configuration interface for service task requirements. It is divided into two main sections: 'Integrity' and 'Trustworthiness'.
 In the 'Integrity' section, there is a button 'Add new integrity requirement'. Below it, a configuration line shows 'Data transmission between servicetask1 and servicetask3 - Send and queue message' followed by 'requires an integrity check with Type: HMAC Algorithm: Sha1'.
 In the 'Trustworthiness' section, there is a slider control. Below the slider, it says 'Current value: 6'. To the right of the slider are 'Update' and 'Remove' buttons.

Fig. 9. Defining integrity and trustworthiness requirements for a service task

In SCF, **data confidentiality** is captured through a tick box and a drop-down menu for the selection of the confidentiality level (cf. **Fig. 7**), for inputs and outputs of service tasks. In our case-study, we first identified all confidential data (e.g., the pilot's identification token). Then, we expressed the data confidentiality property on each data flow comprehending at least one confidential datum. This modelling approach was found heavy and error-prone, because confidential data is involved in many flows. SCF should allow specifying data confidentiality on individual datum, rather than on data flows.

Data transmission integrity and **data authenticity** requirements were captured wherever needed with the choice of the *HMAC* construct and the *SHA1* algorithm. The notation was found to be very compact, as illustrated in **Fig. 9**, through the specification of the data flow source and target services, the type of integrity check to be implemented and the supporting algorithm. However, we deplored the lack of support

for the choice of the construct and algorithm. Moreover, it was not possible to capture integrity requirements on control flows (by opposition to data flows).

In SCF, **Trustworthiness** is captured through an input field and an *Add* button (cf. **Fig. 9**). Lack of end-user documentation made the assessment of the capture of trustworthiness more complex than it should have been. We first thought that the initial trustworthiness values of atomic services were automatically assigned some neutral values. It turned out, that the initial trustworthiness values of atomic services needed to be initialised in the marketplace and that could only be done manually in an ad hoc fashion.

In SCF, **Role Based Access Control** (RBAC) allows for the specification of a Role's permission to perform an Action on the selected service; the possible actions are *Full Access*, *Claim*, *Assign* and *Complete*. The RBAC labels were found to be not self-explicit and the tool provided no definition for each of those actions.

In SCF, **Need-to-know** (NtK) allows for the specification of a Role's permission to perform an Action on a Process Variable of the selected service. The possible actions are *Read*, *Write* and *Read/Write*. In our case study, a number of NtK requirements were input. Coverage was good, but modelling work proved itself a long and tedious process. Indeed, input is done task per task, iterating on each applicable role.

Action :

Process Variable :

Role :

NtK Permissions :

	Name	Process Variable	Action	Roles
☐	Perm-usertask7-MET_Request...	MET_Request	write	Non-Flying_Pilot
☐	Perm-usertask7-foobar-write	foobar	write	Non-Flying_Pilot

Fig. 10. Need-to-know specification

Delegation allows for the specification of delegation permissions on user tasks only, involving the definition of a Delegation Type, and when relevant, a Negotiable attribute and a Delegation Depth attribute. The possible delegation types are *None*, *Simple* and *Transfer*, whilst the Negotiable attribute can take its value in {*Yes*, *No*, *Dynamic*}: the labels were found to be not self-explicit and the tool provided no definition for each of those options.

In our SWIM scenario, we needed to model a **segregation** security requirement (cf. AG-SR7 in §3). This specification could not be achieved using SCF.

5.4 A Seamless Transition from Design-Time to Runtime

Our ATM SWIM case-study was planned as a design-time test case only. However, the Aniketos engineering process blurs the frontier between design-time and runtime (cf. §5.2). Thus, working on pure design-time engineering activities with the Aniketos tool suite proved impossible. We therefore briefly assessed a set of runtime tools that were required to perform design-time activities, i.e. tools supporting: (i) the compilation of security properties, (ii) the generation and validation of composition plans, and (iii) the deployment of a composite service. The corresponding feedback is provided

below, but it does not go into as much details as for the aforementioned Aniketos design-time methods and tools.

The **compilation of security properties** is ensured by the Conspec editor. The Conspec Editor allows for the editing of the security properties specified in the Service Composition Framework. Encoding and saving is then done into an eXtensible Mark-up Language (XML) file, for reuse by other Aniketos modules. This additional and redundant format was found to be tedious and a potential source of inconsistencies and regression within the secure service engineering framework. Moreover, it is very complex to use, and the approach does not scale.

The SCF allows for **generating composition plans** for all possible compositions of service components. It is then possible to browse through the plans, as shown in Fig. 11. However, the number of plans rises dramatically fast, and the value of this browsing capability does not extend beyond toy examples, especially with respect to the human cognitive capability to select one composition plan amongst hundreds of options.

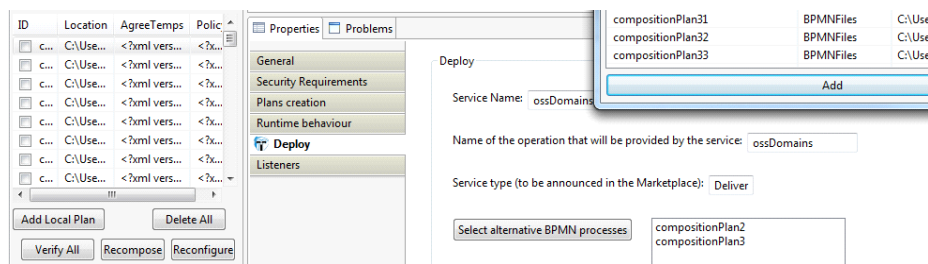


Fig. 11. Generating composition plans and selecting alternatives for deployment.

Generation and **verification of composition plans** allows for the short-listing and sorting of plans according to security properties: trustworthiness, credibility (based on how many security properties have been verified) and validity (based on when the verified security properties are to expire). This capability was not assessed.

Deploying composite services was however quite straightforward, using the built in function for connecting to a service runtime environment (SRE). We were able to use an already operational SRE infrastructure, with an execution engine for running the BPMN process provided by Activiti. The composite service process could be invoked as expected, as a web service.

5.5 Conclusion

The Service Composition Framework was found to be a reliable framework providing a rich set of security properties to work with. Even though the ATM SWIM test case provided us with a well-defined evaluation context, none of the security requirements that we worked on were ATM specific. Therefore SCF seems to be a sound and generic baseline for future industrial exploitation. However, the engineering process with SCF requires service discovery to support security requirements specification at design-time (cf. §5.2): this engineering process issue must absolutely be solved before

considering industrial exploitation. Moreover, SCF suffers from a number of weaknesses that also need to be corrected before considering industrial exploitation, e.g. unclear semantics for some concepts (cf. §5.3), inconsistent integration of satellite tools, and unnecessary technical complexities (cf. §5.4).

6 Conclusion

The ATM SWIM case-study (cf. §23) was specified from the start as a design-time case. Naturally, it focused on the Aniketos Socio-Technical Security Requirements package and on the Aniketos Secure Service Specification and Deployment package (cf. Chapter 4).

The Socio-Technical Security modelling language, methodology and tool were extensively used. The overall assessment was definitely positive, even if some issues were raised with respect to rapid industrial use, but nothing unusual with respect to a research prototype (cf. §4). The main strength of the STS-tool is that it corresponds to a real Air Traffic Management (ATM) community need for a tool to support the elicitation of security requirements in the current phase of System Wide Information Management (SWIM) development.

The Aniketos Secure Service Specification and Deployment package prompts stronger reservations. This package was found to be a reliable framework providing a rich set of security properties to work with, and therefore seems to be a sound baseline for future industrial exploitation (cf. §5). However, SCF suffers from a number of weaknesses, e.g., unclear semantics for some concepts, fuzzy integration of satellite tools, and unnecessary technical complexities. Moreover, end-users (both Aniketos partners and external ATM and SESAR stakeholders) had difficulties in understanding and applying the overall design-time security engineering process. From our perspective, the Technology Readiness Level (TRL) of this package still requires to be raised before considering industrial application in the ATM domain.

More information on the Aniketos SWIM test case and its detailed evaluation can be found in [15, 16].

Disclaimer

© SESAR JOINT UNDERTAKING, 2014. The content of this section was provided under the responsibilities of the Aniketos Case Study B partners, including NATMIG for the SESAR JOINT UNDERTAKING within the frame of the SESAR programme co-financed by the EU and Eurocontrol. The opinions expressed herein reflect the authors' view only. The SESAR JOINT UNDERTAKING is not liable for the use of any information included herein. Reprint with approval of publisher and with reference to source code only.

References

1. Deliverables of SESAR Project 08.01.04 – “Aeronautical information (AIS Airport Manning, Airport Network, Terrain and Obstacles)”.
2. Deliverables of SESAR Project 08.01.06 – “Information Modelling Meteorological Domain”.
3. Deliverables of SESAR Project 08.03.03 – “Identify and Develop Aeronautical Information ATM Services”.
4. Deliverables of SESAR Project 09.19 – “SWIM Air-Ground Capability”.
5. Deliverables of SESAR Project 09.31 – “Aeronautical databases”.
6. Deliverables of SESAR Project 09.48 – “AIS/MET Serv. & Data Distr”.
7. Deliverables of SESAR Project 13.2.2 – “Aeronautical Information Management subsystem definition”.
8. Deliverables of SESAR Work Package 14 Projects – “System Wide Information Management (SWIM)”.
9. Deliverables of SESAR Project 16.06.02 – “Security support and coordination function”.
10. SESAR Joint Undertaking, “SESAR Factsheet n°02/2010, Business Trajectory / ‘4D’ Trajectory,” 2010. [Online]. Available: http://www.sesarju.eu/sites/default/files/documents/reports/SESAR_Factsheet_4DTrajectory__2_.pdf. [Accessed 06 06 2014].
11. Elda Paja et al., “Final version of the socio-technical security modelling language and tool”, deliverable D1.4, Mai 2013, FP7 ICT Secure and Trustworthy Composite Services (ANIKETOS) project.
12. French national agency for the security of IT systems (ANSSI), “Expression of Needs and Identification of Security Objectives (EBIOS)”, Risk management method, 2010. In French.
13. “CRAMM – the Total Information Security Toolkit,” <http://www.cramm.com/>.
14. M. S. Lund, B. Solhaug, and K. Stølen, “Model-Driven Risk Analysis – The CORAS Approach”. Springer, 2011.
15. Michela D’Errico et al., “Final report on Aniketos applied to industrial case studies”, deliverable D6.4, June 2014, FP7 ICT Secure and Trustworthy Composite Services (ANIKETOS) project.
16. Elke Beck et al., “Results of the final validation and evaluation of the ANIKETOS platform”, deliverable D7.3, June 2014, FP7 ICT Secure and Trustworthy Composite Services (ANIKETOS) project.